

Data Architecture CA4: E-Commerce Data Pipeline Report

Matthew Riddell

May 08, 2026

1 Environment and Tools

1.1 Introduction

The aim of this project was to develop an end-to-end data pipeline similar to that of a modern e-commerce platform. This pipeline was designed to collect product information from Shopify APIs so that it could be processed across multiple database systems. The pipeline involved the ingestion, transformation and storage of product data using MongoDB, PostgreSQL, Neo4j and XML.

This project was developed using DkIT's Linux-based server environment. This environment was created to simulate a real-world enterprise system. Similar to real-world data engineering workflows, the project was designed and executed remotely instead of locally on a personal machine. All components of the pipeline, including MongoDB, PostgreSQL, Neo4j and Python, were accessed remotely through a secure SSH connection.

The requirement to use a remote server introduced several important considerations such as session management, resource allocation and infrastructure security. These are all common considerations in professional data engineering environments where applications and databases are hosted remotely.

1.2 SSH Connection and Remote Access

Secure Shell (SSH) was used to connect to the remote Linux server. The connection was established using the following command:

```
ssh username@10.200.172.60
```

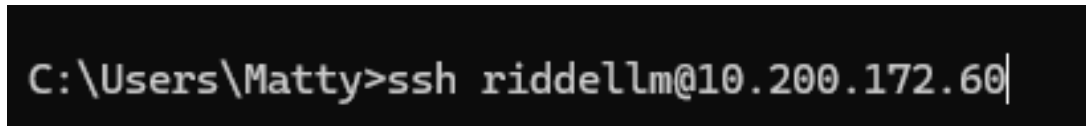


Figure 1: SSH Connection to the Remote Server

Once authenticated, the user gains access to the Linux shell environment allowing the user to execute tasks in the environment. SSH enables encrypted communication between the client and the server, this ensures that sensitive information cannot be intercepted.

This approach mirrors that of cloud based platforms such as Microsoft Azure and Amazon Web Service which all host remote servers for businesses and organizations alike, all interactions between client and server are likewise securely encrypted, preventing any outside threats from obtaining sensitive information.

1.3 Tmux Session Management

To improve the workflow, `tmux` was used as a terminal for executing commands on the remote server, `tmux` is designed in a way that a session can be split into multiple windows allowing for multiple terminal sessions within a single SSH connection (Red Hat, 2022).

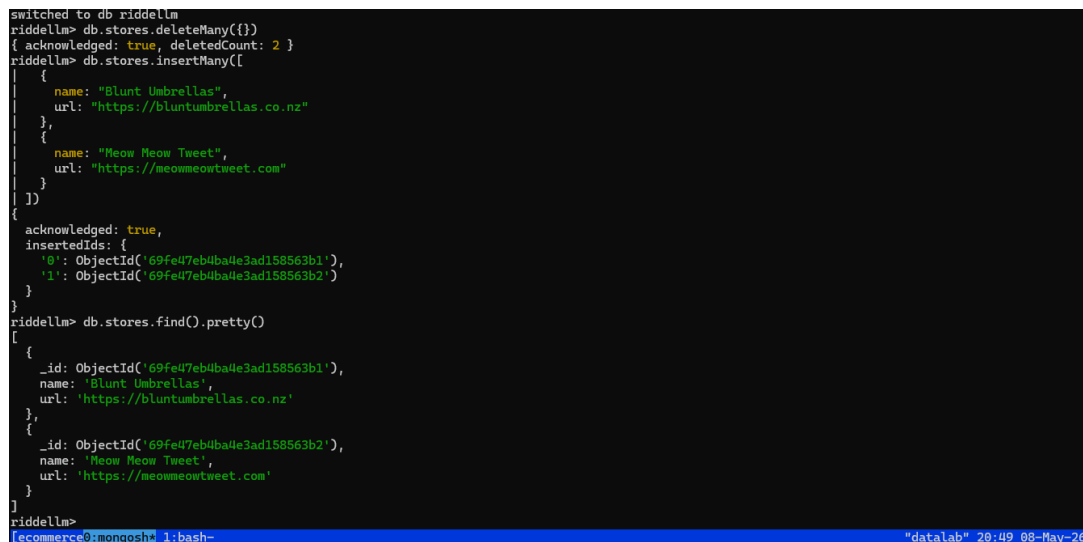
The following command was used to create a new `tmux` session:

```
tmux new -s ecommerce
```

Within this `tmux` session, multiple windows were created for different purposes:

1.3.1 MongoDB Window

A MongoDB shell called `mongosh` was created for creating and handling the MongoDB database and for executing MongoDB queries.



```
switched to db riddellm
riddellm> db.stores.deleteMany({})
{ acknowledged: true, deletedCount: 2 }
riddellm> db.stores.insertMany([
  {
    name: "Blunt Umbrellas",
    url: "https://bluntumbrellas.co.nz"
  },
  {
    name: "Meow Meow Tweet",
    url: "https://meowmeowtweet.com"
  }
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69fe47eb4ba4e3ad158563b1'),
    '1': ObjectId('69fe47eb4ba4e3ad158563b2')
  }
}
riddellm> db.stores.find().pretty()
[
  {
    _id: ObjectId('69fe47eb4ba4e3ad158563b1'),
    name: 'Blunt Umbrellas',
    url: 'https://bluntumbrellas.co.nz'
  },
  {
    _id: ObjectId('69fe47eb4ba4e3ad158563b2'),
    name: 'Meow Meow Tweet',
    url: 'https://meowmeowtweet.com'
  }
]
riddellm>
[ecommerce@mongosh# 1:bash- "dataLab" 20:49 08-May-26]
```

Figure 2: MongoDB Shell Running in tmux

1.3.2 Python Window

A python environment `ipython3` was created for writing code remotely using Nano and also for executing scripts, and aswell for directory management across the whole environment (Linuxize, 2019).

1.3.3 PostgreSQL Window

A PostgreSQL shell `psql` was created for creating the PostgreSQL database and for executing PostgreSQL queries.

Tmux allows for the creation of new windows by pressing:

`Ctrl + B + c`

```
riddellm@datalab: ~  
Inserted  
Deodorant Stick  
Inserted  
Skin Cream  
Inserted  
Cleansing Oil  
Inserted  
Facial Bar  
Inserted  
Spot Serum  
Inserted  
Deodorant Cream  
Inserted  
Face Cleanser  
Inserted  
Repair Balm  
Inserted  
Face Oil  
Inserted  
Shampoo Bar  
Inserted  
Insect Repellent  
Inserted  
Deodorant Cream  
Inserted  
Vegan Lip Balm  
Inserted  
Body Soap  
Inserted  
Body Soap  
Inserted  
Finished loading products.  
riddellm@datalab:~/ecommerce_pipeline$  
[ecommerce0:mongosh- 1:bash*
```

Figure 3: IPython3 Running in tmux

```
psql (16.13 (Ubuntu 16.13-0ubuntu0.24.04.1))  
Type "help" for help.  
  
riddellm=> |  
[ecommerce0:mongosh- 1:psql*
```

Figure 4: PostgreSQL Shell Running in tmux

This creates a new numbered tab on the bottom of the screen.

Users can switch between windows using:

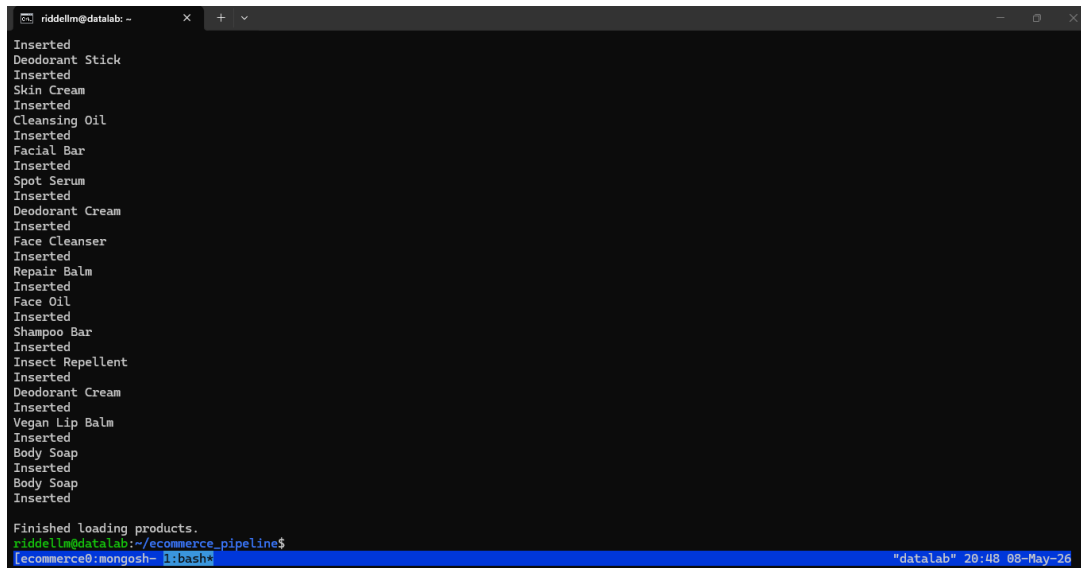
Ctrl + B + window number

This setup enabled the switching between multiple concurrent sessions without having to open or close the existing terminal and also without requiring multiple SSH sessions, enabling workflow efficiency. Tmux proves to be invaluable in situations where multiple sessions are needed. For example, whenever the user needs to switch between databases and Python scripts during development.

1.4 Python Environment and Dependency Management

A Python virtual environment was created to handle directory management as well as for writing and executing scripts on the remote server.

python3



```
riddellm@datalab: ~  
Inserted  
Deodorant Stick  
Inserted  
Skin Cream  
Inserted  
Cleansing Oil  
Inserted  
Facial Bar  
Inserted  
Spot Serum  
Inserted  
Deodorant Cream  
Inserted  
Face Cleanser  
Inserted  
Repair Balm  
Inserted  
Face Oil  
Inserted  
Shampoo Bar  
Inserted  
Insect Repellent  
Inserted  
Deodorant Cream  
Inserted  
Vegan Lip Balm  
Inserted  
Body Soap  
Inserted  
Body Soap  
Inserted  
Finished loading products.  
riddellm@datalab:~/ecommerce_pipeline$  
[ecommerce0:mongosh- 1:bash* "dataLab" 20:48 08-May-26]
```

Figure 5: Python Virtual Environment

Due to server level restrictions, certain packages could not be installed, the session had to rely on existing preinstalled packages and in some cases needed to use certain packages that were permitted to be installed or using workarounds. In this case for the PostgreSQL, the package `psycopg2-binary` (Gregorio, n.d.) needed to be installed using:

```
pip install --break-system-packages psycopg2-binary
```

The following Python Libraries were used throughout this project:

- **Pymongo:** MongoDB connection
- **Requests:** API calls

- **Psycopg2**: PostgreSQL connection
- **Configparser**: Configuration management
- **xml.etree.ElementTree**: XML report generation

The Tmux environment was set up so that each session was in a separate window:



Figure 6: Tmux sessions

- Window 0: Mongosh connected to MongoDB
- Window 1: PSQL connected to PostgreSQL
- Window 2: IPython3 running inside tmux

2 MongoDB: Data Ingestion and Querying

MongoDB was set up and connected to the mongosh session via

```
mongosh --host 10.200.172.54 -u yourusername
```

The initial stores collection was created in the mongosh session based on the URLs of the JSON endpoints.

Then to connect it to the python session the *mongodb.conf* file was created, this is the database server connection configuration file. Then a script in python using nano was created, this file is called *load_products.py* and was used to populate the collections in the MongoDB database.

2.1 Collections Design

Two collections were created in MongoDB for this project.

2.1.1 Stores Collection

This collection stores metadata about each Shopify store including the store name and store URL.

The two stores used in this project were:

- Blunt Umbrellas
- Meow Meow Tweet

```
ridellm> db.stores.find().pretty()
[
  {
    _id: ObjectId('69fe47eb4ba4e3ad158563b1'),
    name: 'Blunt Umbrellas',
    url: 'https://bluntumbrellas.co.nz'
  },
  {
    _id: ObjectId('69fe47eb4ba4e3ad158563b2'),
    name: 'Meow Meow Tweet',
    url: 'https://meowmeowtweet.com'
  }
]
```

Figure 7: Stores Collection

2.1.2 Products Collection

The products collection stores detailed product information retrieved from Shopify APIs.

Each product contains fields such as:

- Product ID
- Title
- Vendor
- Price
- Variants
- Store Reference

MongoDB was used as it was designed entirely around using JSON documents, the nested nature of JSON data storage makes it ideal for ingesting Shopify API data which also uses JSON making it perfectly compatible with little adaptation required.

2.2 Data Ingestion Process

Data was retrieved from the Shopify store's `/products.json` API endpoint which contains a JSON showing store and product information.

The ingestion script executed following these steps:

1. Reads store URLs from MongoDB
2. Sends HTTP requests using `requests.get()`
3. Parses the fetched JSON response
4. The script then iterates through the list of products in the JSON file.
5. The script then adds metadata fields: Store name and Reference ID
6. The data parsed from the from each product in the list is then inserted into the MongoDB database.


```
riddellm@datalab: ~  
Inserted  
Deodorant Stick  
Inserted  
Skin Cream  
Inserted  
Cleansing Oil  
Inserted  
Facial Bar  
Inserted  
Spot Serum  
Inserted  
Deodorant Cream  
Inserted  
Face Cleanser  
Inserted  
Repair Balm  
Inserted  
Face Oil  
Inserted  
Shampoo Bar  
Inserted  
Insect Repellent  
Inserted  
Deodorant Cream  
Inserted  
Vegan Lip Balm  
Inserted  
Body Soap  
Inserted  
Body Soap  
Inserted  
Finished loading products.  
riddellm@datalab:~/ecommerce_pipeline$  
[ecommerce0:mongosh- 1:bash*
```

Figure 8: MongoDB Ingestion Script

2.3 Error Handling and Duplication Handling

In order to maintain data integrity, duplicate insertion errors were handled in the Ingestion script using exception handling:

```
riddellm@datalab: ~  
Duplicate item ignored  
Deodorant Stick  
Duplicate item ignored  
Skin Cream  
Duplicate item ignored  
Cleansing Oil  
Duplicate item ignored  
Facial Bar  
Duplicate item ignored  
Spot Serum  
Duplicate item ignored  
Deodorant Cream  
Duplicate item ignored  
Face Cleanser  
Duplicate item ignored  
Repair Balm  
Duplicate item ignored  
Face Oil  
Duplicate item ignored  
Shampoo Bar  
Duplicate item ignored  
Insect Repellent  
Duplicate item ignored  
Deodorant Cream  
Duplicate item ignored  
Vegan Lip Balm  
Duplicate item ignored  
Body Soap  
Duplicate item ignored  
Body Soap  
Duplicate item ignored  
Finished loading products.  
riddellm@datalab:~/ecommerce_pipeline$  
[ecommerce0:mongosh- 1:python3*
```

Figure 9: Duplicate Handling in Python

Using the Store name and the Product ID, a compound unique index was created to ensure that the same product from the same store could not be inserted more than once.

```
products_collection.create_index(
```

```
[('store', 1), ('handle', 1)],  
unique=True  
)
```

This compound Index was used to prevent duplicate product entries, maintain consistency and to enforce uniqueness across the database.

2.4 Pagination Handling

Shopify APIs usually limit the number of products that get returned in a single API request. This means that some stores may require more than one API request in order to receive the full product catalogue. For really large product datasets, pagination is an important technique that enables data pipeline to receive all the data.

In this project the selected Shopify stores returned all of their data in a single API call, this meant that for this project, pagination was not necessary. But for future adaptations of this project, Pagination may be considered in order to adapt to Stores with larger catalogues.

Shopify APIs typically limit the number of products returned in a single API request. This means that stores with larger product catalogues may require multiple requests in order to retrieve the complete dataset. Pagination is therefore an important consideration in large-scale data ingestion pipelines.

It is an important consideration nevertheless as pagination ensures that all data is retrieved maintaining data integrity, consistency and preventing data loss (Nordic APIs, 2019).

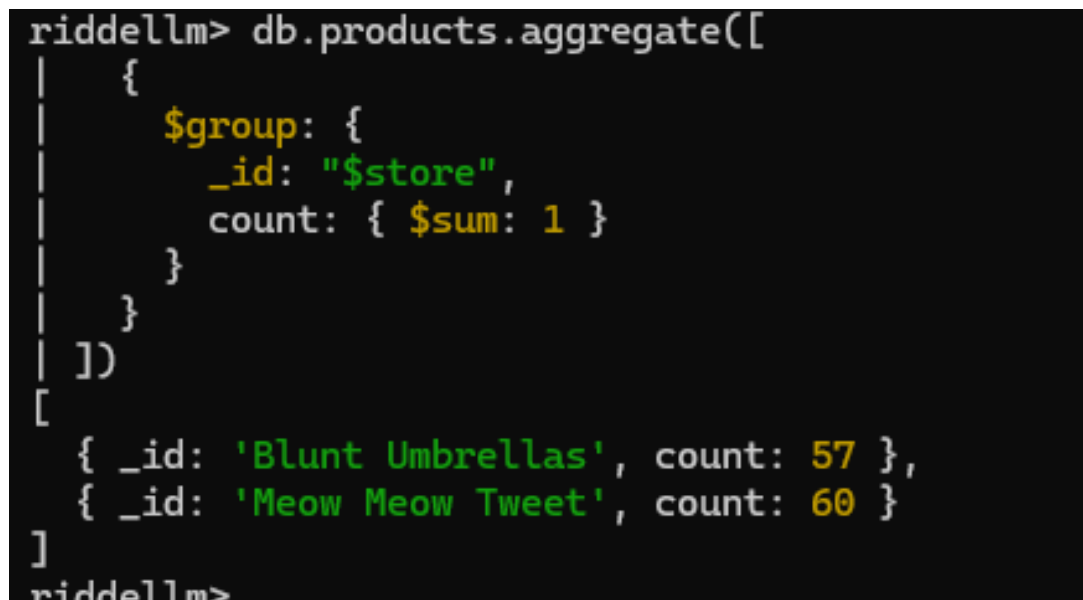
2.5 MongoDB Queries

2.5.1 Query 1: Count the number of products per store

```
db.products.aggregate([
  {
    $group: {
      _id: "$store",
      count: { $sum: 1 }
    }
  }
])
```

Interpretation:

This query groups products by store and counts how many products each store contains.



```
riddellm> db.products.aggregate([
|   {
|     $group: {
|       _id: "$store",
|       count: { $sum: 1 }
|     }
|   }
| ])
|
| [
|   { _id: 'Blunt Umbrellas', count: 57 },
|   { _id: 'Meow Meow Tweet', count: 60 }
| ]
riddellm>
```

Figure 10: MongoDB Query 1 Output

2.5.2 Query 2: Average product price per Store

```
db.products.aggregate([
```

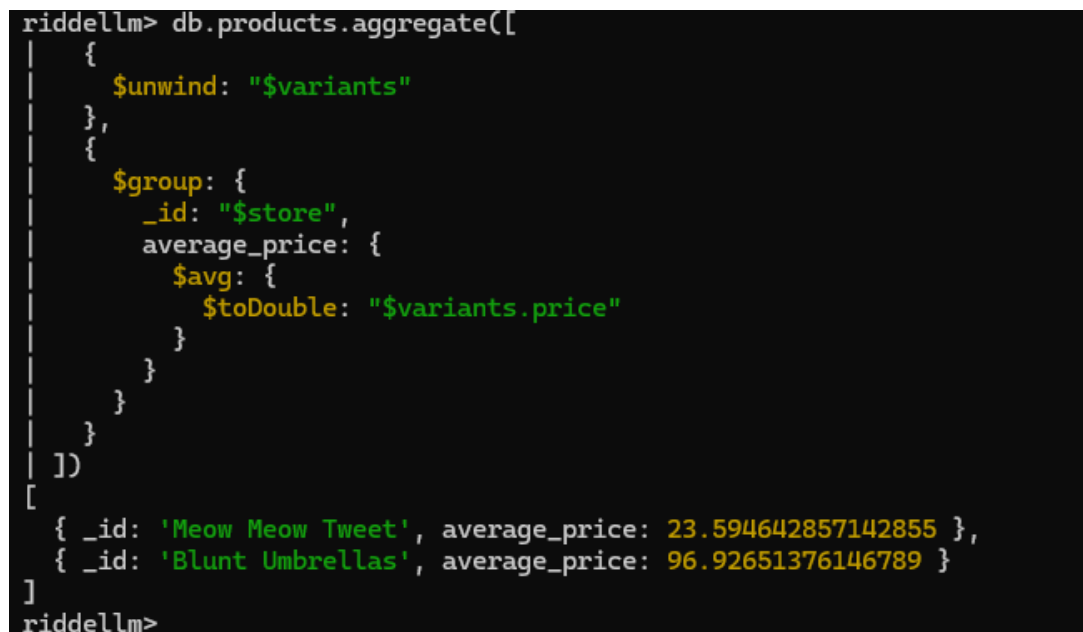
```

{
  $unwind: "$variants"
},
{
  $group: {
    _id: "$store",
    average_price: {
      $avg: {
        $toDouble: "$variants.price"
      }
    }
  }
}
}
])

```

Interpretation:

This query calculates the average product price for each store.



```

riddellm> db.products.aggregate([
  {
    $unwind: "$variants"
  },
  {
    $group: {
      _id: "$store",
      average_price: {
        $avg: {
          $toDouble: "$variants.price"
        }
      }
    }
  }
])
[
  { _id: 'Meow Meow Tweet', average_price: 23.594642857142855 },
  { _id: 'Blunt Umbrellas', average_price: 96.92651376146789 }
]
riddellm>

```

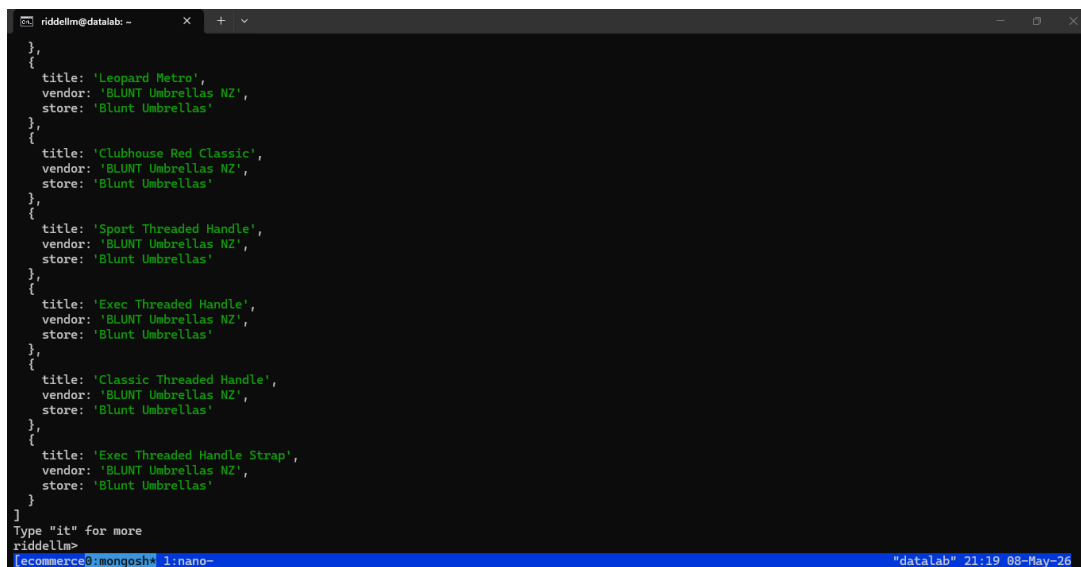
Figure 11: MongoDB Query 2 Output

2.5.3 Query 3: Displaying each product but only showing the title, vendor and store name

```
db.products.find(  
  {},  
  {  
    title: 1,  
    vendor: 1,  
    store: 1,  
    _id: 0  
  }  
)
```

Interpretation:

This query retrieves displays each product but only showing the title, vendor and store name.



```
},  
{  
  title: 'Leopard Metro',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
},  
{  
  title: 'Clubhouse Red Classic',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
},  
{  
  title: 'Sport Threaded Handle',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
},  
{  
  title: 'Exec Threaded Handle',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
},  
{  
  title: 'Classic Threaded Handle',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
},  
{  
  title: 'Exec Threaded Handle Strap',  
  vendor: 'BLUNT Umbrellas NZ',  
  store: 'Blunt Umbrellas'  
}  
]  
Type "it" for more  
riddellm>
```

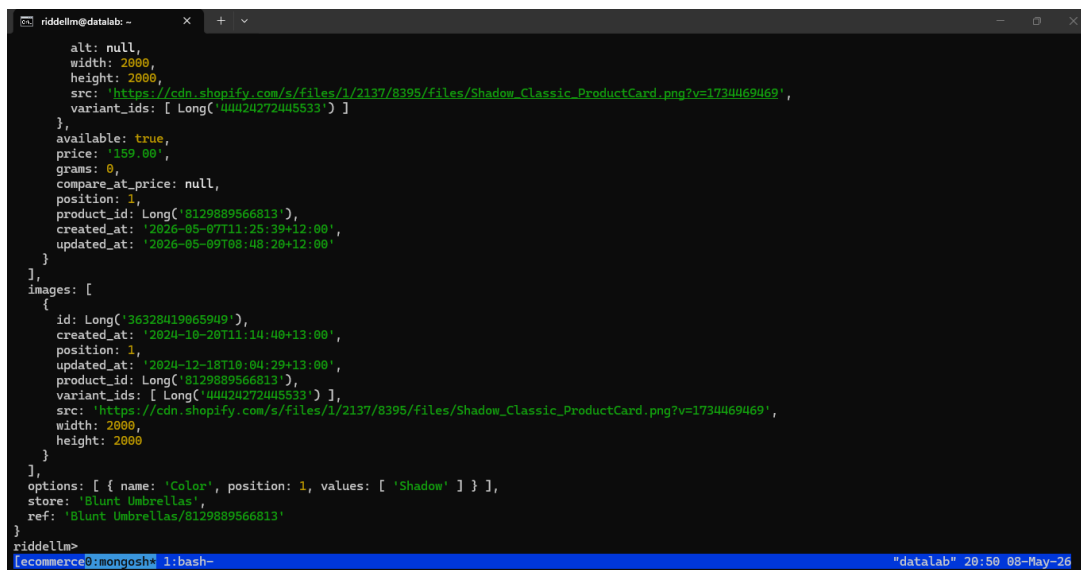
Figure 12: MongoDB Query 3 Output

2.5.4 Query 4: Find products that have tags

```
db.products.find({
  tags: { $exists: true, $ne: [] }
})
```

Interpretation:

This query retrieves each product that has a tag.



```
alt: null,
width: 2000,
height: 2000,
src: 'https://cdn.shopify.com/s/files/1/2137/8395/files/Shadow_Classic_ProductCard.png?v=1734469469',
variant_ids: [ Long('44424272445533') ]
},
available: true,
price: '159.00',
grams: 0,
compare_at_price: null,
position: 1,
product_id: Long('8129889566813'),
created_at: '2026-05-07T11:25:39+12:00',
updated_at: '2026-05-09T08:48:20+12:00'
}
],
images: [
  {
    id: Long('36328419065949'),
    created_at: '2024-10-20T11:14:40+13:00',
    position: 1,
    updated_at: '2024-12-18T10:04:29+13:00',
    product_id: Long('8129889566813'),
    variant_ids: [ Long('44424272445533') ],
    src: 'https://cdn.shopify.com/s/files/1/2137/8395/files/Shadow_Classic_ProductCard.png?v=1734469469',
    width: 2000,
    height: 2000
  }
],
options: [ { name: 'Color', position: 1, values: [ 'Shadow' ] } ],
store: 'Blunt Umbrellas',
ref: 'Blunt Umbrellas/8129889566813'
}
riddellm>
[ecommerce@mongosh 1: bash- "dataLab" 20:50 08-May-26
```

Figure 13: MongoDB Query 4 Output

3 PostgreSQL: Schema and Transactions

In the python session, the file *mongo_to_pg.py* was created. This was used to migrate the MongoDB data to SQL.

3.1 Relational Schema Design

Two relational tables were created in PostgreSQL.

3.1.1 Stores Table

```
CREATE TABLE IF NOT EXISTS stores (  
    id SERIAL PRIMARY KEY,  
    name TEXT UNIQUE  
);
```

3.1.2 Products Table

```
CREATE TABLE IF NOT EXISTS products (  
    id SERIAL PRIMARY KEY,  
    store_id INTEGER REFERENCES stores(id),  
    product_id BIGINT,  
    title TEXT,  
    vendor TEXT,  
    product_type TEXT,  
    price NUMERIC  
);
```

A SERIAL Primary Key was used in the Products table in order to automatically create unique ids in each row serially.

A foreign key was created using `store_id` which was placed in the products table so that each product references the correct store, this ensures referential integrity.

The following Data Types were used in the fields:

- TEXT: For flexible string storage
- NUMERIC: For prices, to ensure accurate numerical values
- BIG NT: For the Shopify Product IDs

3.2 Transaction Management

The MongoDB to PostgreSQL script used transactions in order to maintain atomicity of each commit to the PostgreSQL database.

```
pg_conn.autocommit = False
```

If any data insertions were to be interrupted or if the data insertion were to fail, the transaction would be rolled back.

```
pg_conn.rollback()
```

Otherwise the data would be committed to the database.

```
pg_conn.commit()
```

This makes sure that each data insertion is atomized, that is to say, it is indivisible, and is committed as a whole and never partially. This also maintains data consistency and Integrity in the database.

3.3 SQL Queries

3.3.1 Query 1: Select all Stores

```
SELECT * FROM stores;
```

Interpretation:

This query displays all stores in the database.


```
riddellm=> SELECT * FROM stores;
id |      name
----+-----
 1 | Blunt Umbrellas
58 | Meow Meow Tweet
(2 rows)
```

Figure 14: SQL Query 1 Output

3.3.2 Query 2: Select the first five products in the database

```
SELECT * FROM products LIMIT 5;
```

Interpretation:

This query displays the first five products in the database.

```
riddellm=> SELECT * FROM products LIMIT 5;
id | store_id | product_id |      title      | vendor          | product_type | price
----+-----+-----+-----+-----+-----+-----
 1 |      1 | 8129889566813 | Classic UV Shadow | BLUNT Umbrellas NZ | Umbrella | 159.0
 2 |      1 | 8129216348253 | Classic UV Lilac Haze | BLUNT Umbrellas NZ | Umbrella | 159.0
 3 |      1 | 8076988514397 | Metro UV Lilac Haze | BLUNT Umbrellas NZ | Umbrella | 139.0
 4 |      1 | 8076988612701 | Metro UV Shadow | BLUNT Umbrellas NZ | Umbrella | 139.0
 5 |      1 | 7994246824029 | Moss Metro | BLUNT Umbrellas NZ | Umbrella | 139.0
(5 rows)

riddellm=>
[ecommerce0:mongosh 1:psql* 2:bash-
```

Figure 15: SQL Query 2 Output

3.3.3 Query 3: Product Count Per Store

```
SELECT
    s.name,
    COUNT(*) AS product_count
FROM products p
JOIN stores s
ON p.store_id = s.id
GROUP BY s.name;
```

Interpretation:

This query counts how many products belong to each store.

```
ridde1lm=> SELECT
    s.name,
    COUNT(*) AS product_count
FROM products p
JOIN stores s
ON p.store_id = s.id
GROUP BY s.name;
```

name	product_count
Meow Meow Tweet	60
Blunt Umbrellas	57

(2 rows)

Figure 16: SQL Query 3 Output

3.3.4 Query 4: Average Product Price

```
SELECT
    s.name,
    AVG(p.price) AS average_price
FROM products p
JOIN stores s
ON p.store_id = s.id
GROUP BY s.name;
```

Interpretation:

This query calculates the average product price for each store.

```

riddellm=> SELECT
    s.name,
    AVG(p.price) AS average_price
FROM products p
JOIN stores s
ON p.store_id = s.id
GROUP BY s.name;

```

name	average_price
Meow Meow Tweet	20.000000000000000000
Blunt Umbrellas	94.9998245614035088

(2 rows)

Figure 17: SQL Query 4 Output

4 Neo4j: Graph Modelling and Queries

Using Neo4J, a graph model was designed to visualize the relationships between stores, products and vendors.

4.1 Node Labels

The following Nodes were created which represent each object:

- Store
- Product
- Vendor

4.2 Relationship Types

The following relationships were established between each node:

SELLS: Store - Product

Each Store sells a Product

MADE_BY: Product - Vendor

Each product is provided by a vendor, in the case of multi platform and universal stores such as amazon or eBay, the store may differ from the vendor itself, hence the different relationship.

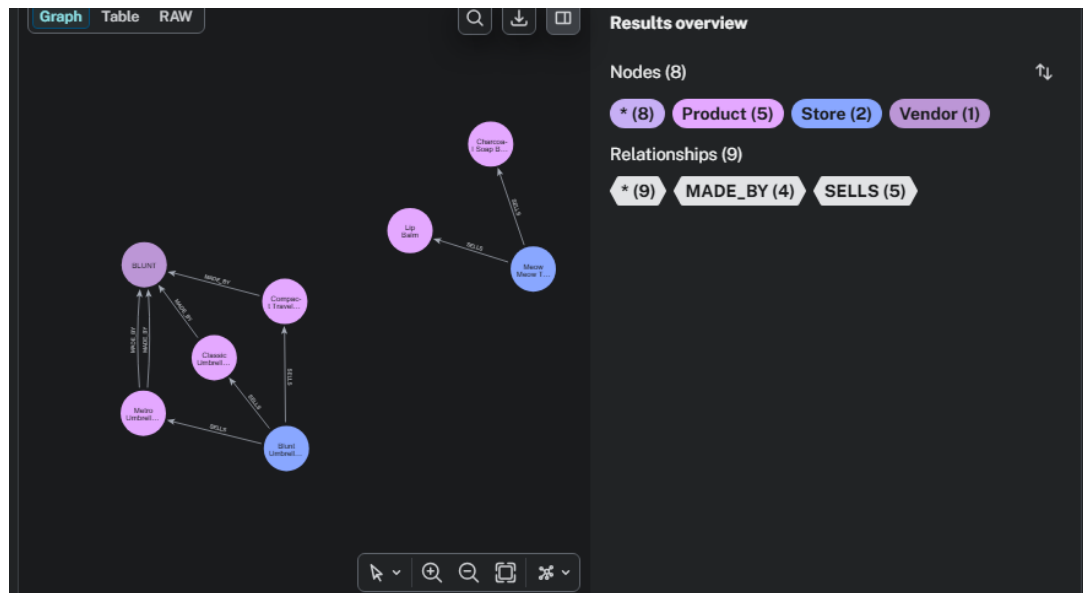


Figure 18: Neo4j Graph Model

Graph databases were chosen as they more efficiently represent relationships between entities, this enables fast traversal queries that would otherwise require a complex join or multiple joins to traverse multiple tables, as is the case for relational databases like SQL.

n	r	m
Blunt Umbrellas	SELLS	Metro Umbrella
Metro Umbrella	SELLS	Blunt Umbrellas
Blunt Umbrellas	SELLS	Classic Umbrella
Classic Umbrella	SELLS	Blunt Umbrellas
Blunt Umbrellas	SELLS	Compact Travel Umbrella
Compact Travel Umbrella	SELLS	Blunt Umbrellas
Meow Meow Tweet	SELLS	Charcoal Soap Bar
Charcoal Soap Bar	SELLS	Meow Meow Tweet

Figure 19: Neo4j Table

4.3 Cypher Queries

4.3.1 Query 1: Products sold by each store

```
MATCH (s:Store)-[:SELLS]->(p:Product)
RETURN s.name AS store, p.title AS product
```

Interpretation:

This query displays which products are sold by each store.

store	product
"Blunt Umbrellas"	"Metro Umbrella"
"Blunt Umbrellas"	"Classic Umbrella"
"Blunt Umbrellas"	"Compact Travel Umbrella"
"Meow Meow Tweet"	"Charcoal Soap Bar"
"Meow Meow Tweet"	"Lip Balm"

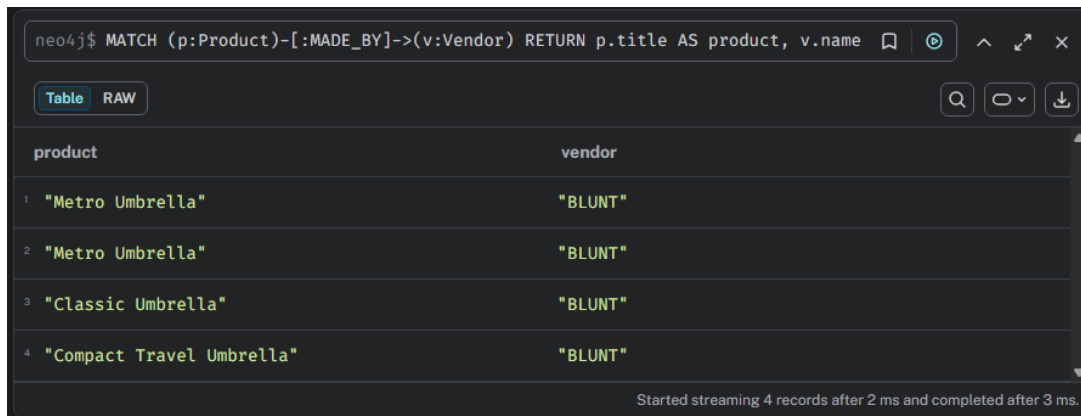
Figure 20: Cypher Query 1 Output

4.3.2 Query 2: Vendors connected to products

```
MATCH (p:Product)-[:MADE_BY]->(v:Vendor)
RETURN p.title AS product, v.name AS vendor
```

Interpretation:

This query identifies which vendor produces each product.



	product	vendor
1	"Metro Umbrella"	"BLUNT"
2	"Metro Umbrella"	"BLUNT"
3	"Classic Umbrella"	"BLUNT"
4	"Compact Travel Umbrella"	"BLUNT"

Started streaming 4 records after 2 ms and completed after 3 ms.

Figure 21: Cypher Query 2 Output

5 XML Report Generation

In the python session, the file *generate_report.py* was created and ran in order to generate an XML report. This file reads the MongoDB config and connects to mongodb, it then parses the JSON using configparser. The XML report is then built using the ElementTree package and outputted as an xml file (Python Software Foundation, n.d.).

The XML report included two queries:

- Product count per store
- Average product price per store

```
<?xml version="1.0" encoding="utf-8"?>
<stores>
  <store name="Blunt Umbrellas">
    <product_count>109</product_count>
    <average_price>96.93</average_price>
  </store>

  <store name="Meow Meow Tweet">
    <product_count>112</product_count>
    <average_price>23.59</average_price>
  </store>
</stores>
```

Figure 22: Generated XML Report

Python’s `ElementTree` module was used to construct the hierarchical XML. Then it was used to iterate through the MongoDB aggregation results and then it wrote the output into an XML report file (Python Software Foundation, n.d.).

The XML output is well-formed because a single root element is used. The XML structure contains proper nesting of elements with parent and child element indentation. The valid encoding method is specified in the head of the file using UTF-8.

6 Infrastructure and Security Reflection

For this kind of configured environment, Linux is perfect as it is stable, secure and widely used in enterprise server environments, especially for server hosting and cloud computing solutions. This is because Linux is lightweight and highly configurable. Being open source and regularly updated, Linux supports a wide variety of technologies, languages and platforms making it modular and customization for any kind of solution a business requires. Linux provides excellent support for database systems and Python development (Ha, 2025).

A suitable RAID configuration for this project would be RAID 1 as it provides redundancy

through disk mirroring. This ensures data remains intact whenever one of the disks gets damaged or lost. Data durability is really important in the event of hardware failure and RAID 1 protects the database from this scenario (Western Digital, n.d.).

A UPS (Uninterruptible Power Supply) is really important for preventing system shutdowns in the event of power outages or blackouts. Unexpected power failures can result in data corruptions or transaction interruptions which threatens data integrity in the database. Furthermore, a power outage can damage hardware too. UPS prevents this by maintaining a backup power supply that can safely power down the system after important transactions have been committed, maintaining data integrity and protecting hardware from damage (Wikipedia Contributors, 2019).

Important credentials were stored in configuration files with restricted permissions:

```
chmod 600 mongodb.conf
```

SSH key authentication is more secure than password authentication as it uses cryptographic key pairs instead of plain text passwords. That is to say, that the data is encrypted in a two key pair, a public key and a private key, and only have both keys can a user access the encrypted data (Cloudflare, 2023).

7 Challenges and Reflection

Many challenges were encountered and resolved throughout the course of this project.

7.1 Server Availability

On many occasions, DkIT server and DkIT VPN access was unresponsive due to server maintenance during weekends, as a result, project work needed to be carefully conducted during weekdays in order to progress in the project.

7.2 Learning Curve

Navigating and using new technologies such as Tmux and Nano in particular required using specific key combinations in order to execute commands, edit files, and navigate pages in the terminal. This was eventually overcome as the project progressed.

7.3 API Limitations

Some Shopify APIs returned large nested JSON structures that required some adaptation to the ingestion script in order to accurately parse and ingest the data into MongoDB.

7.4 Duplicate Data

Duplicate insertions occurred initially when testing the data ingestion script, so the database needed to be dropped a few times and the script was adapted to handle this by using a compound unique index in the product collection.

7.5 Data Type Conversion

MongoDB stored prices as string values, this caused aggregation errors when executing the average price count queries, to resolve this the `$toDouble` operator was used to convert the price values into doubles, then they were able to execute flawlessly.

7.6 PostgreSQL Transactions

Transaction failures occasionally occurred during testing. Rollback functionality ensured consistency was maintained.

On occasion, transaction errors occurred due to connection issues, this initially required

dropping the tables to fix, but with the addition Rollback in the Mongo to PostgreSQL script, the transaction was protected from partial commits, removing the need for dropping the tables and starting again.

7.7 Reflection

This project allowed for the development of a data pipeline from end to end, this demonstrated the strengths and weaknesses of multiple database systems and technologies.

MongoDB provided a level of flexibility as it is designed around handling unstructured JSON data.

PostgreSQL provided a strong relational database with a strong structure suitable for relational integrity and transactional consistency.

Neo4J provided an efficient relationship traversal and graph visualization of the data.

Overall this project proved invaluable for learning about various Database management systems technologies and how they can be used to develop pipelines through remote SSH sessions. Furthermore, this project also proved invaluable for teaching about data integrity and data security.

Bibliography

Cloudflare. (2023). *What is ssh? — secure shell (ssh) protocol*. Retrieved from <https://www.cloudflare.com/learning/access-management/what-is-ssh/>

Gregorio, F. D. (n.d.). *psycopg2: psycopg2 - python-postgresql database adapter*. PyPI. Retrieved from <https://pypi.org/project/psycopg2/>

Ha, D. (2025, August). *What is a linux server? everything you need to know*. LogicMonitor. Retrieved from <https://www.logicmonitor.com/blog/why-linux-is>

-a-popular-choice-for-servers

Linuxize. (2019, June 30). *How to use nano, the linux command line text editor*. Linuxize.

Retrieved from <https://linuxize.com/post/how-to-use-nano-text-editor/>

Nordic APIs. (2019, October 17). *Everything you need to know about api pagination*. Retrieved from <https://nordicapis.com/everything-you-need-to-know-about-api-pagination/>

Python Software Foundation. (n.d.). *xml.etree.elementtree — the elementtree xml api — python 3.10.2 documentation*. Python Documentation. Retrieved from <https://docs.python.org/3/library/xml.etree.elementtree.html>

Red Hat. (2022). *A beginner's guide to tmux*. Red Hat. Retrieved from <https://www.redhat.com/en/blog/introduction-tmux-linux>

Western Digital. (n.d.). *What is raid storage levels 0, 1, 5, 10*. Retrieved from <https://www.westerndigital.com/en-ie/solutions/raid>

Wikipedia Contributors. (2019, May 13). *Uninterruptible power supply*. Wikipedia. Retrieved from https://en.wikipedia.org/wiki/Uninterruptible_power_supply